Wireshark를 이용한 transport layer traffic 분석

학과: 소프트웨어학과 학번: 201920792 이름: 유승민

1. 개요

온라인 멀티플레이 게임인 'starcraft', 'league of legends' 두 가지로 UDP와 TCP packet을 캡처한다. 캡처한 UDP와 TCP를 분석하는데 게임 네트워크 기법을 조금 가미해 내용을 심화한다. 'starcraft'의 경우 LoginMe Hamachi application을 이용해 가설 사설망(VPN)을 개설하고 LAN 게임을 플레이 한다. 가설 사설망에서 packet을 캡처하면 결과가 깔끔하다.

2. 관찰 방법

2021.05.12 11:30 자택에서 PC로 게임 starcraft를 실행 후 관찰. LAN/ONLINE 두 네트워크 환경에서 관측 후 12:30에 관측 종료. LAN의 경우 LogMein Hamachi 어플리케이션으로 만든 가설 사설망을 이용. ONLINE의 경우 BLIZZARD에서 제공하는 server를 이용. I/O Graph, IPv4 statistics 기능을 사용해 packet을 분석.

2021.05.12 12:30 동일한 환경에서 League of Legends 실행 후 관찰. 이더넷에서 capture 진행. 칼바람 모드를 한 판 플레이. 이 후 League of Legends client 로그인 과정을 다시 캡처후 1:00에 Wireshark 종료,

3. Traffic 분석

1) Transport layer protocol traffic 분석

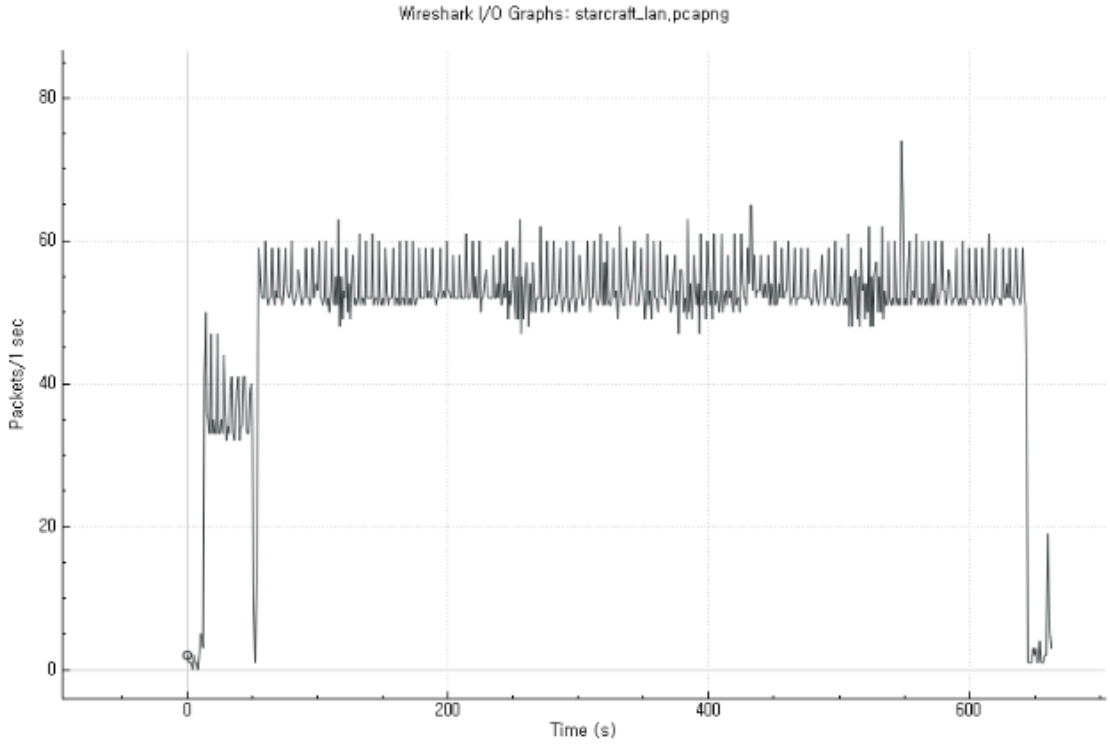

그림 1 <starcraft LAN game I/O Graphs>

그림 1 은 LogMein Hamachi 프로그램을 이용해 가설 사설망 (VPN)을 만들어 게임 starcraft 에서 LAN game 을 플레이했을 때의 I/O Graphs 다. 가설 사설망에서만 packet 을 capture 했으며, 모든 packet 이 가설 사설망 속 친구와 주고받은 것들이다(몇 개 예외가 있을 수는 있다.). LAN game 의 상대는 내 친구다. 관찰 환경의 변화는 LAN server 접속, 친구가 개설한 사설 방에 입장, 게임 플레이, 게임 종료까지의 단계를 거쳤다. LAN server 에 접속했을 때 traffic 이 높다가 게임을 시작하는 순간에 traffic 이 한 순간 급감한다. 그 후 본격적인 게임이 시작되며 이때부터 traffic 이 꾸준하게 높다. 게임이 끝나고 게임 결과창이 나왔을 때 traffic 이 급감하며 이 때 capture 를 중단했다.

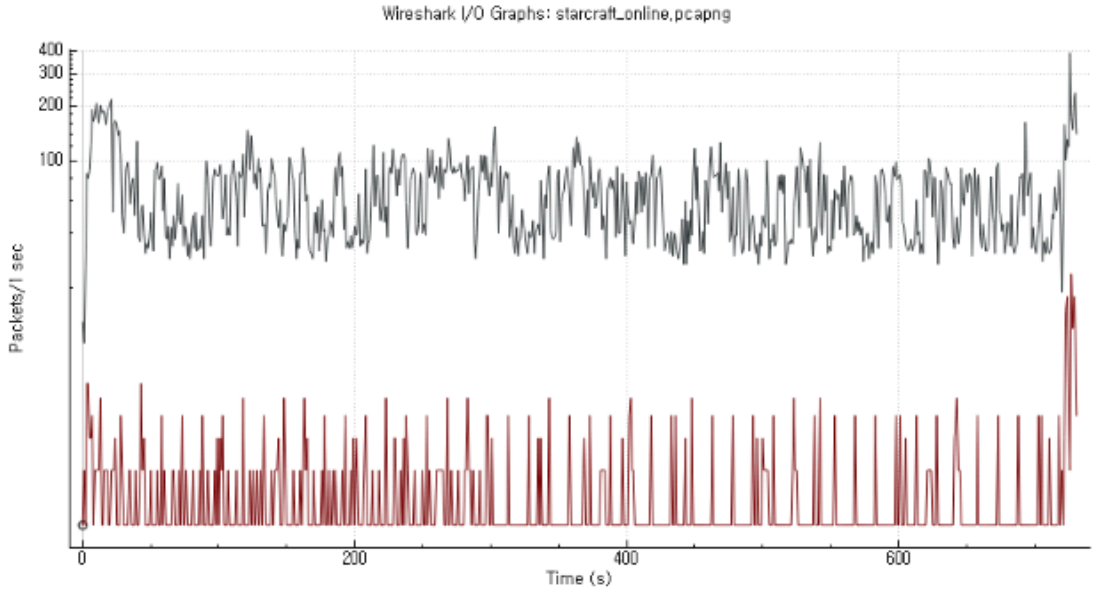

그림 2 <starcraft Online game I/O Graphs>

그림 2 는 게임회사 Blizzard 에서 제공하는 starcraft korea server 에 접속해 게임을 한 판 플레이 했을 때의 I/O Graphs(Log scale)다. 검은색 그래프는 모든 packet, 빨간색 그래프는 게임 상대와 교류한 packet 에 관한 각각의 traffic 을 나타낸 거다. LAN game 과 달리 친구와 직접 교류하는 packet 의 양이 1~4 개 정도이다. 서로 교류하는 packet 들이 LAN 과 달리 블리자드가 제공하는 서버를 통하고 있음을 알 수 있다.

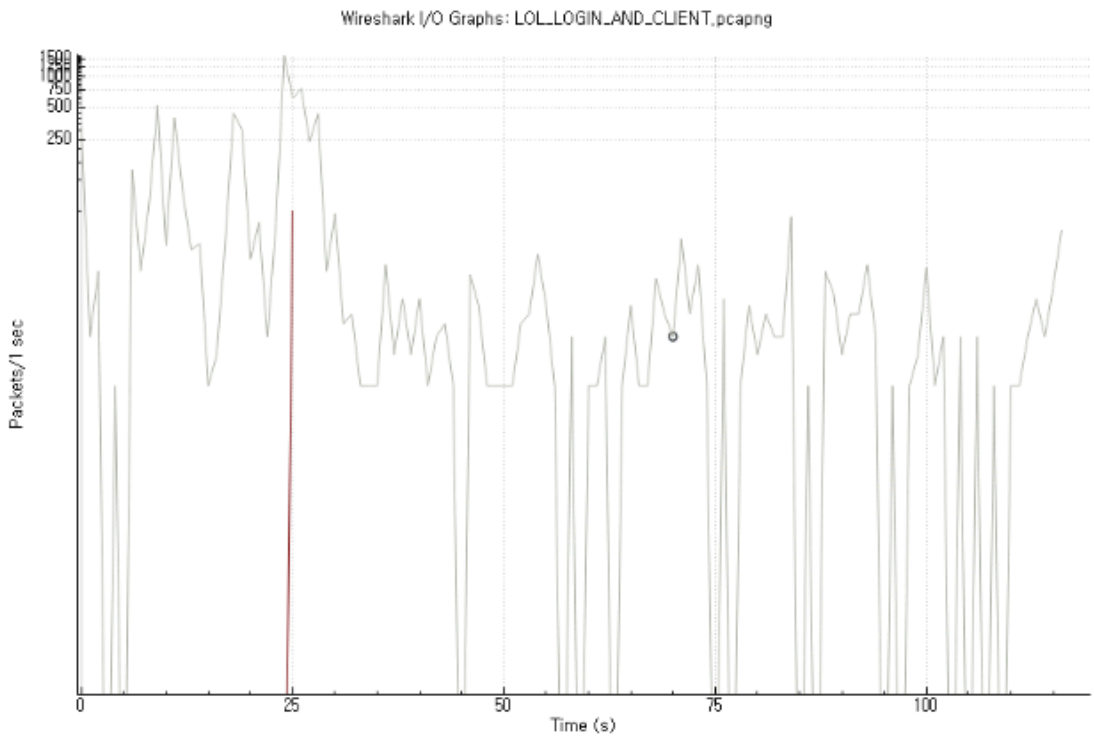

그림 3 <League of Legends Login>

그림 3 은 게임 League Of Legends 에 로그인 할 때의 I/O Graphs(Log scale)이다. 옅은 회색 그래프는 모든 packet 이동량이고, 빨간색 그래프는 로그인에 관한 TCP 중 하나의 stream 을 필터링했을 때 packet 이동량이다. 로그인 순간에 packet 이동량이 가장 많음을 알 수 있다.

2) UDP 분석

[주: 관찰된 packet 들 중에서 UDP 를 사용하는 서비스 사례를 찾고,

해당 서비스(protocol) 의 목적, message 종류 및 형식 등을 조사해서 기술, 이때, 서버스가 굳이

사용자를 대상으로 하는 것이 아니어도 좋으나 DNS 는 제외함]

644.149212	25.60.5.11	25.59.210.211
644.181265	25.60.5.11	25.59.210.211
644.228456	25.60.5.11	25.59.210.211
644.274180	25.60.5.11	25.59.210.211
644.304387	25.60.5.11	25.59.210.211
644.351480	25.60.5.11	25.59.210.211
644.398516	25.60.5.11	25.59.210.211
644.429875	25.60.5.11	25.59.210.211
644.477474	25.60.5.11	25.59.210.211
644.524034	25.60.5.11	25.59.210.211
644.553783	25.60.5.11	25.59.210.211

그림 1 <starcraft LAN game UDP packet 전송 주기>

그림 1 은 친구 쪽에서 내 쪽으로 보내는 UDP packet 의 time, source ip, destination ip 들이다. 0.04s~0.05s 의 주기로 UDP packet 을 전송하고 있다. IP 주소가 25 로 시작하는 이유는 LoginMe hamachi 에서 배정하는 내부 IP 이기 때문이다.

Topic / Item	Count	Average	Min Val	Max Val	Rate (ms)	Percent
Source IPv4 Addresses	33015				0.0498	100%
25.60.5.11	16847				0.0254	51.03%
25.59.210.211	16168				0.0244	48.97%

그림 2 <starcraft LAN game IPv4 statics>

그림 2 는 친구와 내가 각각 전송한 packet 의 통계다. 25.60.5.11 이 친구의 ip 이고 25.59.210.211 가 나의 ip 이다. 자료를 조금 찾아본 결과 UDP 패킷 전송량이 비슷한 이유는 스타크래프트가 '락 스탭(lock step)'기법을 사용했기 때문이다. 스타크래프트는 실시간 전략 시뮬레이션 게임으로 알려져 있지만 사실상 그 원리는 턴 기반이라고 한다. 스타크래프트에는 턴 레이트라는 것이 존재하며 각각의 턴 마다 플레이어가 내리는 일종의 명령들을 수집하여 나머지 플레이어들에게 해당 턴에서 행해진 모든 명령을 공유한다. 이런 동기화 기법 덕분에 스타크래프트는 클라이언트 간의 동기화가 거의 완벽하다. 유닛 하나가 건물 사이에 끼어 갈팡질팡 하는 모습조차 모든 클라이언트에게 동일하게 보여진다. UDP 가 일정 주기마다 전송되고 내 종단 시스템에서 송신한 패킷 수와 상대 종단 시스템으로부터 수신한 패킷 수가 비슷한 이유는 스타크래프트가 턴 기반 기법을 사용해서 그렇다. 내 쪽에서 송신한 패킷 수보다 수신한 패킷 수가 적다. 상대가 전송한 UDP 패킷에는 미약한 손실이 있음을 알 수 있다.

```
User Datagram Protocol, Src Port: 6112, Dst Port: 6112
    Source Port: 6112
    Destination Port: 6112
    Length: 37
    Checksum: 0xad1a [unverified]
    [Checksum Status: Unverified]
    [Stream index: 3]
  > [Timestamps]
    UDP payload (29 bytes)
Data (29 bytes)
    Data: 0801121700000000499a13007c2c7d2c02000000373244d4ce075c5a00
    [Length: 29]
```

그림 3 <starcraft LAN game UDP Packet Header>

그림 3 은 client 끼리 주고받은 UDP 의 헤더다. UDP payload 에는 사용자가 일정 주기(약 0.04s)동안 입력한 키보드, 마우스 인풋이 저장되어 있을 것이다.

17420	242.335222	192.168.219.106	59.153.41.83	UDP	83	6112 → 35471 Len=41
17422	242.343269	59.153.41.83	192.168.219.106	UDP	83	35471 → 6112 Len=41
17426	242.412206	192.168.219.106	59.153.41.83	UDP	83	6112 → 35471 Len=41
17428	242.423689	59.153.41.83	192.168.219.106	UDP	83	35471 → 6112 Len=41
17433	242.505362	192.168.219.106	59.153.41.83	UDP	83	6112 → 35471 Len=41
17435	242.516370	59.153.41.83	192.168.219.106	UDP	83	35471 → 6112 Len=41
17440	242.584404	192.168.219.106	59.153.41.83	UDP	83	6112 → 35471 Len=41
17441	242.588325	192.168.219.106	59.153.41.83	UDP	80	6112 → 35471 Len=38

그림 4 <starcraft ONLINE game UDP packet>

그림 4 는 online server 에서 플레이 했을 때의 UDP packet 을 캡쳐한 거다. LAN 의 경우 client 끼리 패킷을 주고 받았는데, online game 의 경우 server 를 통해서 packet 을 주고받는다. ip 주소 59.153.41.83의 위치를 찾아보니 일본이였다. 또한 LAN 일 때보다 DATA 의 크기가 약 10 bytes 쯤 높다. Packt 전송 시간이 뒤죽박죽이긴 하지만 대략 0.07s 정도의 주기를 가진다. LAN 보다 주기가 길다.

3) TCP 분석

- 5000 - 5500 UDP (리그 오브 레전드 게임 클라이언트)
- 8393 - 8400 TCP (패치 프로그램 및 마에스트로)
- 2099 TCP (PVP.Net)
- 5223 TCP (PVP.Net)
- 5222 TCP (PVP.Net)
- 80 TCP (HTTP 연결)
- 443 TCP (HTTPS 연결)
- 8088 UDP 및 TCP (관전자 모드)

그림 1 <port list used by League of Legend>

그림 1 은 게임 'League of Legend'에서 사용하는 port 의 number 와 용도를 나타내고 있다. 롤 클라이언트에 로그인하고 접속할 때 443 번 포트를 사용하는 tcp 의 스트림을 필터링해 packet 을 capture했다.

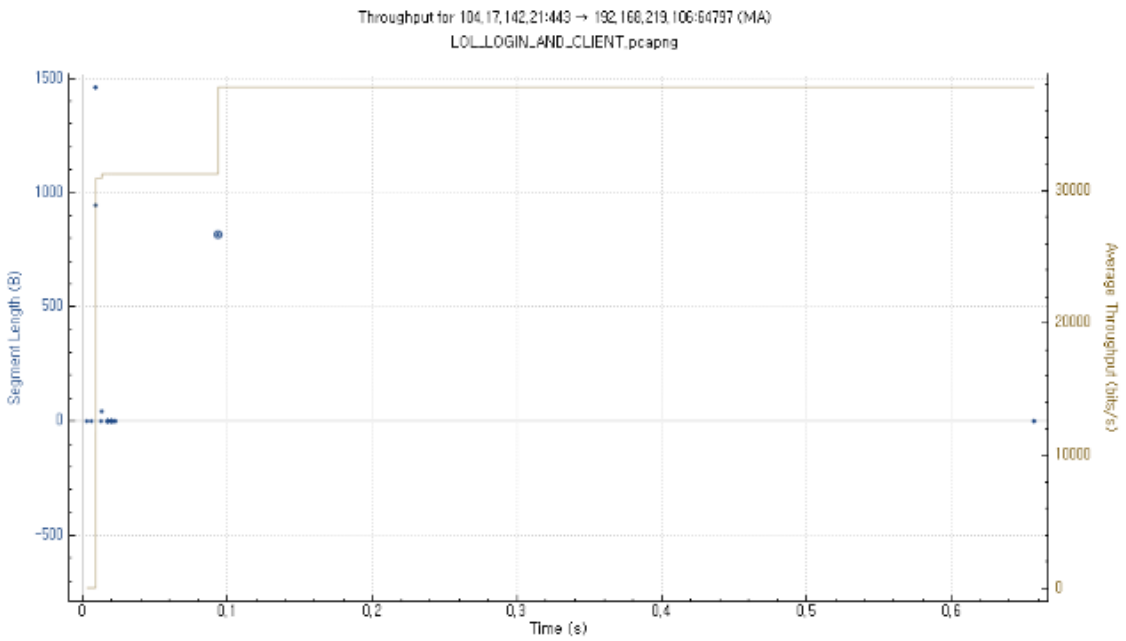

그림 2 <tcp stream graph->time sequence(tcptrace)>

그림 2 는 패킷 이동량을 파란 점으로 나타낸 것이다. League of Legends 로그인 과정에는 TLS 기법이 들어가 있다. 그래프의 가장 좌측에서 TCP connection setup 과 TLS handshake 가 이루어진다. 이 시점에 패킷 이동량이 집중되어 있다. 가장 우측 패킷이 connection 을 종료하는 패킷이다.

```
Transmission Control Protocol, Src Port: 64797, Dst Port: 443, Seq: 0, Len: 0
    Source Port: 64797
    Destination Port: 443
    [Stream index: 85]
    [TCP Segment Len: 0]
    Sequence Number: 0    (relative sequence number)
    Sequence Number (raw): 2635264270
    [Next Sequence Number: 1    (relative sequence number)]
    Acknowledgment Number: 0
    Acknowledgment number (raw): 0
    0111 .... = Header Length: 28 bytes (7)
  > Flags: 0x002 (SYN)
    Window: 64240
    [Calculated window size: 64240]
    Checksum: 0x925c [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  > Options: (8 bytes), Maximum segment size, No-Operation (NOP), No-Operation (NOP), SACK permitted
  > [Timestamps]
```

그림 3-1 <TCP segment header in TCP connection setup>

그림 3-1 은 TCP connection setup 의 첫 번째 세그먼트로 Flages 가 SYN 이다. client 에서 server 에게 연결을 요청하는 세그먼트다. SYN 세그먼트라고도 불리며, SYN 은 연결을 시작하고 회선을 개설하는 용도로 쓰인다. Destination Port 가 443 인데, League of Legends 에서 명시했듯이 443 포트는 HTTPS 연결에 쓰인다. Server 에 보내는 첫 packet 이기 때문에 relative sequence

number 가 0 이다. sequence number 는 송신할 data 의 첫 바이트를 가리킨다. ACK 가 0 인데, 현재 server 로부터 수신한 패킷이 없기 때문에 그렇다. ACK 는 송신받기를 기대하는 데이터의 첫 바이트를 가리킨다. Header Length 사 28bytes 인데, 이는 8bytes 만큼의 option 이 존재함을 가리킨다. Option 이 없을 때 Header Length 의 길이는 20bytes 이다. Options 을 살펴보면 Maximum segment size 와 SACK permitted 가 있다. No-Operation(NOP)는 옵션들을 구분 짓는 구분선이다. Maximum segment size(MSS)는 전송할 수 있는 segment 의 최대 사이즈를 제안하는 것이다. 이더넷에서 관측을 진행하고 있어서 MSS 는 1460 이다. 이더넷의 MTU 1500 에서 IPv4 헤더인 20 bytes, TCP 헤더인 20 bytes 를 뺀 값이다. SACK permitted 는 손실된 segment 에 의해 ACK 를 다시 보낼 때, 손실된 후 받은 packet 의 정보를 보내는 기법을 사용하는 옵션이다. 이렇게 되면 손실된 segment 부터 packet 을 재전송할 필요 없이, 손실된 segment 만 재전송하면 된다. Checksum은 TCP 헤더 데이터를 포함한 세그먼트 전체에 대하여 계산한 값이다. 오류체크에 쓰인다. Urgent Pointer 는 긴급히 처리해야 할 필요가 있는 데이터의 마지막 바이트의 위치를 나타낸다. 값이 현재로썬 0 이다. Window 는 수신측이 받을 수 있는 데이터 사이즈를 수신측에서 송신쪽으로 전송하는 값이다.

```
Transmission Control Protocol, Src Port: 443, Dst Port: 64797, Seq: 0, Ack: 1, Len: 0
    Source Port: 443
    Destination Port: 64797
    [Stream index: 85]
    [TCP Segment Len: 0]
    Sequence Number: 0    (relative sequence number)
    Sequence Number (raw): 141134665
    [Next Sequence Number: 1    (relative sequence number)]
    Acknowledgment Number: 1    (relative ack number)
    Acknowledgment number (raw): 2635264271
    0111 .... = Header Length: 28 bytes (7)
  > Flags: 0x012 (SYN, ACK)
    Window: 65535
    [Calculated window size: 65535]
    Checksum: 0xd063 [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  > Options: (8 bytes), Maximum segment size, No-Operation (NOP), No-Operation (NOP), SACK permitted
  > [SEQ/ACK analysis]
  > [Timestamps]
```

그림 3-2 <TCP segment header in TCP connection setup>

그림 3-2 는 TCP connection setup 의 두 번째 세그먼트로 Flages 가 SYN, ACK 이다. Server 가 client 에 보내는 패킷의 세그먼트이며, 첫 번째 패킷에 대한 응답 패킷의 세그먼트이다. SYN 은 위와 마찬가지로 연결을 시작하고 회선을 개설하는 용도로 쓰이며, ACK 는 client 가 보내온 패킷을 받았음을 알리는 동시에 다음에 받기를 기대하는 데이터의 첫 번째 바이트를 가리킨다. 이 TCP segment header 에서 눈에 띄는 것은 Option 으로 MSS 값이 1400 이다. server 의 IP 주소는 104.17.142.21로 위치를 검색해보면 미국에 위치하고 있다. 이 위치는 League of Legends 의 본사가 위치한 캘리포니아주 산타 모니카와는 멀리 떨어져 있다. 아무래도 IP 주소 104.17.142.21는 League of Legends 에서 다양한 지역에 배치한 여러 server 중 하나인가보다. 그 지역의 server 가 존재하는 인터넷 망의 MTU 는 1440 임을 알 수 있다.

```
Transmission Control Protocol, Src Port: 64797, Dst Port: 443, Seq: 1, Ack: 1, Len
    Source Port: 64797
    Destination Port: 443
    [Stream index: 85]
    [TCP Segment Len: 0]
    Sequence Number: 1    (relative sequence number)
    Sequence Number (raw): 2635264271
    [Next Sequence Number: 1    (relative sequence number)]
    Acknowledgment Number: 1    (relative ack number)
    Acknowledgment number (raw): 141134666
    0101 .... = Header Length: 20 bytes (5)
  > Flags: 0x010 (ACK)
    Window: 64240
    [Calculated window size: 64240]
    [Window size scaling factor: -2 (no window scaling used)]
    Checksum: 0x9254 [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  > [SEQ/ACK analysis]
  > [Timestamps]
```

그림 3-3 <TCP segment header in TCP connection setup>

그림 3-3 은 TCP connection setup 의 세 번째 세그먼트로 Flags 가 ACK 이다. 이 ACK 를 보냄으로써 연결이 이루어지고 데이터가 오가기 시작한다.

```
Transmission Control Protocol, Src Port: 64797, Dst Port: 443, Seq: 46977, Ack: 4724, Len: 0
    Source Port: 64797
    Destination Port: 443
    [Stream index: 85]
    [TCP Segment Len: 0]
    Sequence Number: 46977    (relative sequence number)
    Sequence Number (raw): 2635311247
    [Next Sequence Number: 46978    (relative sequence number)]
    Acknowledgment Number: 4724    (relative ack number)
    Acknowledgment number (raw): 141139389
    0101 .... = Header Length: 20 bytes (5)
  > Flags: 0x011 (FIN, ACK)
    Window: 63541
    [Calculated window size: 63541]
    [Window size scaling factor: -2 (no window scaling used)]
    Checksum: 0x9254 [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  > [Timestamps]
```

그림 4-1 <TCP segment header in TCP connection 종료>

그림 4-1 은 TCP connection 종료의 첫 번째 세그먼트로 Flags 가 FIN, ACK 이다. FIN 은 세션 연결을 종료시킬 때 사용되며 더 이상 전송할 데이터가 없음을 나타낸다.

```
Transmission Control Protocol, Src Port: 443, Dst Port: 64797, Seq: 4724, Ack: 46977, Len: 0
    Source Port: 443
    Destination Port: 64797
    [Stream index: 85]
    [TCP Segment Len: 0]
    Sequence Number: 4724    (relative sequence number)
    Sequence Number (raw): 141139389
    [Next Sequence Number: 4724    (relative sequence number)]
    Acknowledgment Number: 46977    (relative ack number)
    Acknowledgment number (raw): 2635311247
    0101 .... = Header Length: 20 bytes (5)
  > Flags: 0x010 (ACK)
    Window: 65535
    [Calculated window size: 65535]
    [Window size scaling factor: -2 (no window scaling used)]
    Checksum: 0x32f8 [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  > [SEQ/ACK analysis]
  > [Timestamps]
```

그림 4-2 <TCP segment header in TCP connection 종료>

그림 4-2 은 TCP connection 종료의 두 번째 세그먼트로 Flags 가 ACK 이다. client 로부터 FIN, ACK 패킷을 받은 server 눈 client 에게 ACK 패킷을 보낸 후 443 번 포트와 연결되 있는 어플리케이션을 종료한다.

```
Transmission Control Protocol, Src Port: 443, Dst Port: 64797, Seq: 4724, Ack: 46978, Len: 0
    Source Port: 443
    Destination Port: 64797
    [Stream index: 85]
    [TCP Segment Len: 0]
    Sequence Number: 4724    (relative sequence number)
    Sequence Number (raw): 141139389
    [Next Sequence Number: 4725    (relative sequence number)]
    Acknowledgment Number: 46978    (relative ack number)
    Acknowledgment number (raw): 2635311248
    0101 .... = Header Length: 20 bytes (5)
  > Flags: 0x011 (FIN, ACK)
    Window: 65535
    [Calculated window size: 65535]
    [Window size scaling factor: -2 (no window scaling used)]
    Checksum: 0x32f6 [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  > [SEQ/ACK analysis]
  > [Timestamps]
```

그림 4-3 <TCP segment header in TCP connection 종료>

그림 4-3 은 TCP connection 종료의 세 번째 세그먼트로 Flags 가 FIN, ACK 이다. 443 번 포트와 연결된 어플리케이션이 종료되면 server 에서 client 에게 보내는 패킷이다.

```
Transmission Control Protocol, Src Port: 64797, Dst Port: 443, Seq: 46978, Ack: 4725, Len: 0
    Source Port: 64797
    Destination Port: 443
    [Stream index: 85]
    [TCP Segment Len: 0]
    Sequence Number: 46978    (relative sequence number)
    Sequence Number (raw): 2635311248
    [Next Sequence Number: 46978    (relative sequence number)]
    Acknowledgment Number: 4725    (relative ack number)
    Acknowledgment number (raw): 141139390
    0101 .... = Header Length: 20 bytes (5)
  > Flags: 0x010 (ACK)
    Window: 63541
    [Calculated window size: 63541]
    [Window size scaling factor: -2 (no window scaling used)]
    Checksum: 0x9254 [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  > [SEQ/ACK analysis]
  > [Timestamps]
```

그림 4-4 <TCP segment header in TCP connection 종료>

그림 4-2 은 TCP connection 종료의 네 번째 세그먼트로 Flags 가 ACK 이다. server 로부터 FIN, ACK packet 을 받았을 때 보내는 응답 패킷이다.

```
Transmission Control Protocol, Src Port: 443, Dst Port: 64797, Seq: 1461, Ack: 518, Len: 1460
    Source Port: 443
    Destination Port: 64797
    [Stream index: 85]
    [TCP Segment Len: 1460]
    Sequence Number: 1461    (relative sequence number)
    Sequence Number (raw): 141136126
    [Next Sequence Number: 2921    (relative sequence number)]
    Acknowledgment Number: 518    (relative ack number)
    Acknowledgment number (raw): 2635264788
    0101 .... = Header Length: 20 bytes (5)
  > Flags: 0x010 (ACK)
    Window: 65535
    [Calculated window size: 65535]
    [Window size scaling factor: -2 (no window scaling used)]
    Checksum: 0x1bab [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  > [SEQ/ACK analysis]
  > [Timestamps]
    TCP payload (1460 bytes)
    [Reassembled PDU in frame: 4134]
    TCP segment data (1460 bytes)
```

그림 5

<TCP segment header including application data>

그림 5 는 server 로부터 client 에게 보애논 packet 이다. TCP connection setup 과정에서 server 에게 client 의 MSS 가 1460 임을 알렸고, 이에 맞춰 서버가 1460 bytes 만큼의 데이터를 client 에게 보냈다. 이 패킷의 relative sequence number 는 1461 이고, nect sequence number 는 2921 로 relative sequence number 에 TCP segment data 의 크기를 더한 값이다.

```
Transmission Control Protocol, Src Port: 64797, Dst Port: 443, Seq: 603, Ack: 3908, Len: 14000
    Source Port: 64797
    Destination Port: 443
    [Stream index: 85]
    [TCP Segment Len: 14000]
    Sequence Number: 603    (relative sequence number)
    Sequence Number (raw): 2635264873
    [Next Sequence Number: 14603    (relative sequence number)]
    Acknowledgment Number: 3908    (relative ack number)
    Acknowledgment number (raw): 141138573
    0101 .... = Header Length: 20 bytes (5)
  > Flags: 0x010 (ACK)
    Window: 64357
    [Calculated window size: 64357]
    [Window size scaling factor: -2 (no window scaling used)]
    Checksum: 0x9240 [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  > [SEQ/ACK analysis]
  > [Timestamps]
    TCP payload (14000 bytes)
    [Reassembled PDU in frame: 4142]
    TCP segment data (14000 bytes)
```

그림 6 <TCP segment header including application data>

그림 6 은 application data 를 가지고 있는 패킷의 TCP segment header 이다. TCP segment data 14000bytes 인데, 이는 server 의 MSS 인 1400bytes 의 10 배다. application data 를 가진다는 점 외에 connection 단계의 TCP segment header 와 크게 다른 점이 없다.

```
4054 64797 → 443 [ACK] Seq=603 Ack=3908 Win=64357 Len=14000 [TCP segment of a reassembled PDU]
  60 443 → 64797 [ACK] Seq=3908 Ack=2003 Win=65535 Len=0
2854 Application Data [TCP segment of a reassembled PDU]
  60 443 → 64797 [ACK] Seq=3908 Ack=3403 Win=65535 Len=0
  60 443 → 64797 [ACK] Seq=3908 Ack=4803 Win=65535 Len=0
5654 64797 → 443 [ACK] Seq=17403 Ack=3908 Win=64357 Len=5600 [TCP segment of a reassembled PDU]
  60 443 → 64797 [ACK] Seq=3908 Ack=6203 Win=65535 Len=0
2854 64797 → 443 [ACK] Seq=23003 Ack=3908 Win=64357 Len=2800 [TCP segment of a reassembled PDU]
  60 443 → 64797 [ACK] Seq=3908 Ack=7603 Win=65535 Len=0
  60 443 → 64797 [ACK] Seq=3908 Ack=9003 Win=65535 Len=0
5654 64797 → 443 [ACK] Seq=25803 Ack=3908 Win=64357 Len=5600 [TCP segment of a reassembled PDU]
  60 443 → 64797 [ACK] Seq=3908 Ack=10403 Win=65535 Len=0
2854 Application Data [TCP segment of a reassembled PDU]
  60 443 → 64797 [ACK] Seq=3908 Ack=11803 Win=65535 Len=0
2854 64797 → 443 [ACK] Seq=34203 Ack=3908 Win=64357 Len=2800 [TCP segment of a reassembled PDU]
  60 443 → 64797 [ACK] Seq=3908 Ack=13203 Win=65535 Len=0
  60 443 → 64797 [ACK] Seq=3908 Ack=14603 Win=65535 Len=0
5654 64797 → 443 [ACK] Seq=37003 Ack=3908 Win=64357 Len=5600 [TCP segment of a reassembled PDU]
  60 443 → 64797 [ACK] Seq=3908 Ack=17403 Win=65535 Len=0
4405 Application Data
  60 443 → 64797 [ACK] Seq=3908 Ack=20203 Win=65535 Len=0
  60 443 → 64797 [ACK] Seq=3908 Ack=23003 Win=65535 Len=0
```

그림 6-2 <TCP pacet>

그림 6-2 는 client 가 sever 에게 data 를 전송하고, server 는 client 에게 ACK 패킷을 보내는 과정을 캡처한 것이다. client 가 server 에게 14000 bytes 의 data 를 전송하자 server 는 1400bytes 씩 나눠 받으며 ACK 패킷을 전송하고 있다. server 가 보내는 패킷의 ACK number 가 1400 씩 증가하고 있다.

```
Transmission Control Protocol, Src Port: 50902, Dst Port: 443, Seq: 3181, Ack: 1499, Len: 1
    Source Port: 50902
    Destination Port: 443
    [Stream index: 13]
    [TCP Segment Len: 1]
    Sequence Number: 3181    (relative sequence number)
    Sequence Number (raw): 2346159183
    [Next Sequence Number: 3182    (relative sequence number)]
    Acknowledgment Number: 1499    (relative ack number)
    Acknowledgment number (raw): 1471303007
    0101 .... = Header Length: 20 bytes (5)
  > Flags: 0x010 (ACK)
    Window: 64240
    [Calculated window size: 64240]
    [Window size scaling factor: -1 (unknown)]
    Checksum: 0xf75c [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  > [SEQ/ACK analysis]
  > [Timestamps]
    TCP payload (1 byte)
> Data (1 byte)
```

그림 7 <TCP Keep Alive>

그림 7 은 client 에서 server 에 보내는 TCP packet 이다. 1 bytes 의 DATA 가 포함되 있다. TCP Keepalive 는 일정 시간이 지나면 열결된 세션이 살아있는 확인하기 위해 작은 패킷을 하나 보낸다. client 가 server 와의 연결을 유지하고 싶어서 보내는 패킷이다.

```
TCP        55 [TCP Keep-Alive] 50902 → 443 [ACK] Seq=3181 Ack=1499 Win=64240 Len=1
TCP        66 [TCP Keep-Alive ACK] 443 → 50902 [ACK] Seq=1499 Ack=3182 Win=65535 Len=0 SLE=3181 SRE=3182
```

그림 7-2 <TCP Keep Alive>

그림 7-2 에서 client 가 server 에게 Keepalive packet 을 보내고, 그에 대한 응답으로 server 가 client 에게 ACK packet 을 보내고 있다.

4. 후기

게임 네트워크가 어떻게 구현돼 있는지 간단하게나마 알 수 있어서 즐거웠다. TCP/UDP 기법과 개념에만 집중하지 않고 실제로 어떻게 쓰이는지를 파악하려 해보니까 많은 공부가 됐다. 게임 네트워크 자료가 방대한 탓에 파고 들어도 끝이 없어 난감했다. 깨달은 내용을 기술하자면 실시간 전략 게임 RTS 를 포함해서 다양한 멀티게임은 P2P 모델을 채택하고 있다. 게임이란 특성상 수많은 오브젝트가 움직이는데 그걸 다양한 플레이어들에게 동기화 시키기 위해서라고 한다. UDP 프로토콜을 사용하는 것도 그 이유중 하나다. 스타크래프트와 같은 RTS 게임을 멀티 플레이로 즐기는데 가장 최적화된 네트워크는 LAN 이다. 패킷 분석을 통해 online server 에서 멀티 플레이를 즐기는 경우 server 가 개입함을 알 수 있었다. League of Legends 에 로그인 후

본격적인 게임을 한 판 플레이하며 패킷을 캡처해봤는데, 게임 내용은 starcraft 와 마찬가지로 UDP 패킷으로 교환하고 있었다. 고전 게임인 starcraft 와 현재 가장 유명한 게임인 League of Legends 는 세대 차이가 나지만 네트워크 기법은 비슷했다.

5. 참고 자료

https://m.blog.naver.com/PostView.nhn?blogId=njh0602&logNo=221160824585&proxyReferer=https:%2F%2Fwww.google.com%2F

https://www.ipipipip.net/index.php?ln=ko&menu=ip-address-location-geolocation-lookup-home&ipaddress=59.153.41.83

http://www.ktword.co.kr/abbr_view.php?m_temp1=836

https://leagueoflegends.fandom.com/wiki/Servers

https://www.youtube.com/watch?v=XOU78DkLFJQ